Task 12 – Day-4 design challenge

Impact analysis

The workload is now explicit: 1,000,000 records and 50,000 search requests per hour. That is about

13.89 requests per second on average (50,000 ÷ 3,600). New records arrive only once per day, so reads greatly outnumber updates. Repeatedly scanning the dataset is costly; a one-time or daily index-building cost can be amortized across many requests.

Evaluate original design

Linear search has worst-case O(1,000,000) key comparisons per query and is unsuitable as the primary production strategy at this scale. Recursive binary search needs about $\log_2(1{,}000{,}000) \approx 20$ comparison levels per query, but requires sorted data and careful update maintenance. A hash index has expected O(1) lookup after O(n) build/update work and therefore best matches the read-heavy pattern.

Adaptation/redesign

Redesign around a persistent hash index on registration_number. During the daily intake, validate uniqueness, write the new records, update the index transactionally, and verify index/database consistency. For a production relational database, a unique index (often a B-tree) is also a robust practical choice because the database engine handles persistence, concurrency, recovery and query planning. The assignment-level hash table

models expected constant-time exact lookup; actual deployment should benchmark the database index and workload.

Evidence-based defence

Factor	Observation	Design implication
1,000,000 records	Linear search may require up to one million comparisons; binary search needs about 20 levels; hash lookup offers expected constant-time lookup.	Avoid repeated full scans for large product catalogues.
50,000 searches/hour	Approximately 13.89 searches per second on average.	Choose an efficient lookup method and test realistic workloads.
Daily additions only	Index construction and updates can be batched or performed incrementally.	Balance index update costs with lookup speed.
Preprocessing	Hash table construction takes expected O(n); sorting takes O(n logn).	Include preprocessing costs in system planning.

Trade- offs	Hash tables use extra memory and can have collision- related worst cases.	Consider memory, updates, and database indexing.